

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

TÍTULO DEL DESAFÍO: Paga Diario: Sistema de Gestión de Microcréditos para la Inclusión Financiera en Celendín

Autores:

Arribasplata Rojas Leonardo David *(100% participación)*

Centurión Tingal Kebin Kenjhi *(100% participación)*

Silva Aguirre Gianfranco Alejandro *(100% participación)*

Curso:

Fundamentos de Programación - CIIN1205P

NRC:

226413.16705 (Presencial)

Docente Asesor:

Vasquez Salinas, Jonathan Alexander

LINK DEL ENLACE GIT HUB DEL REPOSITORIO:

<https://github.com/Giango2025/PagaDiario-Celendin.git>

Cajamarca - Perú

2026-1

*Paga Diario: Sistema de Gestión de
Microcréditos para la Inclusión
Financiera en Celendín*

Contenido

I. CAPÍTULO 1.....	4
1.1. Descripción del problema	4
1.2. Análisis del problema	5
1.3. Diagrama de flujo	9
1.4. Pseudocódigo	12
1.5. Código implementado	13
1.6. Pruebas y resultados.....	14
1.7. Reflexión técnica y ética	16
1.8. Evidencias de autoaprendizaje y mejora.....	18
II. ANEXOS	201

I. CAPÍTULO 1.

1.1. Descripción del problema

Contexto del proceso observado

El escenario de estudio se ubica en la provincia de Celendín, región Cajamarca, donde una organización local sin fines de lucro se dedica a la colocación de microcréditos y captación de ahorros bajo la modalidad de recaudo diario denominada “*Paga Diario*”. Esta entidad atiende a comerciantes minoristas y pequeños agricultores que requieren financiamiento rápido para sus actividades productivas. Durante una visita de campo de 30 minutos, se observó que el cobrador (agente de campo) realiza todas las operaciones de manera manual: anota los abonos en cartulinas individuales y en cuadernos de control, calcula los intereses con una calculadora básica y elabora los cronogramas de pago a mano.

Entrevista al encargado

El agente principal manifestó: “*Atiendo unas 30 transacciones al día; cada una me toma cerca de 5 minutos porque debo revisar el cuaderno, calcular el 15% de interés y escribir el comprobante. Al final de la jornada, muchas veces los números no cuadran y los clientes reclaman. Además, los cuadernos se mojan o se pierden, y no tengo un respaldo.*”

Problemas detectados

- **Alta latencia operativa:** el tiempo promedio de registro es de 5 minutos, lo que representa 2,5 horas diarias solo para el registro manual.
- **Errores recurrentes:** el cálculo manual del interés plano del 15% presenta errores en aproximadamente el 20% de las operaciones, generando discrepancias y pérdidas económicas.
- **Falta de control:** los saldos y las deudas no se actualizan en tiempo real; al cierre del día, el efectivo en caja no coincide con los apuntes.
- **Riesgo de pérdida de información:** los soportes físicos (cuadernos y cartulinas) se deterioran por el uso y las condiciones del entorno, y no existe copia de respaldo.
- **Insatisfacción del cliente:** los usuarios esperan largos tiempos para ser atendidos y reciben recibos sin detalle claro, lo que afecta la confianza.

Cuantificación del impacto

- Frecuencia: 30 operaciones diarias.
- Tiempo actual: 5 min/operación → 150 minutos diarios (2,5 horas).
- Porcentaje de error: 20 % de las transacciones con cálculos incorrectos.
- Con la automatización, el tiempo se reduciría a menos de 1 minuto por operación, eliminando los errores aritméticos y liberando al agente para tareas de mayor valor. La productividad mejoraría en un 80 % y los errores operativos disminuirían en un 40 %, según reportes del Ministerio de la Producción.

Diagrama de Ishikawa (análisis de causa-efecto – 6M)

El desglose de las fuentes de variabilidad y fallas del proceso manual se modela con el estándar de las 6M:

- **Mano de obra:** Fatiga cognitiva del operador por la repetición de tareas; errores humanos al transcribir y calcular.

Paga Diario: Sistema de Gestión de Microcréditos para la Inclusión Financiera en Celendín

- **Método:** Ausencia de un procedimiento estandarizado; cada agente usa su propio sistema de anotación; no hay validación cruzada.
- **Materiales:** Dependencia de cuadernos, cartulinas y lapiceros que se deterioran o extravían; falta de formatos preimpresos.
- **Medición:** Descuadres frecuentes entre el efectivo recolectado y los registros manuales; no se lleva un control acumulado de intereses.
- **Maquinaria/tecnología:** Carencia total de un sistema informático; la única herramienta es una calculadora básica sin memoria.
- **Medio ambiente:** Ruido y distracciones en el lugar de trabajo (mercado o calle) que aumentan la probabilidad de equivocaciones.

Justificación de la automatización

La solución digital es altamente relevante para la ingeniería de sistemas porque permite mitigar el riesgo operativo, garantiza la persistencia e integridad de las transacciones financieras y optimiza el tiempo de procesamiento de la información. Transforma un flujo de trabajo vulnerable, propenso a errores y pérdidas, en un sistema auditable, seguro y escalable. Además, proporciona reportes instantáneos que facilitan la toma de decisiones gerenciales.

– **Diagrama de Ishikawa.**



1.2. Análisis del problema

- Generación y selección de soluciones.

Para abordar el problema, el equipo propuso dos alternativas de solución, evaluándolas con criterios técnicos medibles

Alternativa	Descripción	Ventajas	Desventajas	Criterios de evaluación
A1: Aplicación de consola en C#	Sistema estructurado, modular, con menú de 5 opciones, arreglos, archivos de texto y validaciones robustas.	– Bajo costo de desarrollo. – Fácil de implementar y mantener. – No requiere infraestructura adicional. – Permite aplicar todos los conceptos del curso (arreglos, funciones, archivos, validaciones).	– Interfaz solo texto. – Escalabilidad limitada a 100 registros por arreglo estático. – No soporta múltiples usuarios concurrentes.	Simplicidad: alta. Tiempo de desarrollo: corto (2 semanas). Riesgo técnico: bajo. Mantenibilidad: alta.
A2: Aplicación móvil (Android)	App nativa con base de datos SQLite local y sincronización en la nube.	– Mayor accesibilidad y portabilidad. – Interfaz gráfica intuitiva. – Sincronización en tiempo real.	– Alto costo y tiempo de desarrollo (mínimo 3 meses). – Requiere conocimientos de programación móvil (Java/Kotlin). – Dependencia de dispositivos modernos y conectividad a internet. – Mayor complejidad de pruebas.	Simplicidad: baja. Tiempo de desarrollo: largo. Riesgo técnico: alto. Mantenibilidad: media.

Criterios de selección (con ponderación implícita): simplicidad, tiempo de desarrollo, riesgo técnico y mantenibilidad.

Solución elegida: Alternativa A1 (consola en C#) porque es la más viable en el tiempo disponible, cumple con todos los requerimientos funcionales y no funcionales, y permite demostrar claramente los conceptos de programación

estructurada, modularidad, manejo de arreglos, archivos y validaciones, tal como se exige en el curso.

- Tabla IPO completa con entradas, procesos y salidas identificadas.

La tabla siguiente descompone el sistema en entradas, procesos y salidas, con ejemplos concretos extraídos de la realidad observada.

Compon ente	Tipo de elemento	Descripción detallada	Ejemplo de dato real
ENTRA DAS	Datos del cliente	DNI (8 dígitos), nombres completos, apellidos, teléfono móvil (9 dígitos).	74125896, Juan Carlos, Pérez Ramírez, 976543210
	Datos del crédito	Monto capital solicitado, descripción del bien en garantía.	500.00, “Lavadora Samsung”, “http://fotos.com/la vadora.jpg”
	Datos de ahorro	Monto a depositar (opciones discretas: 20, 30, 40 o 50 soles, según política de riesgos).	30.00
	Datos de búsqueda	DNI del cliente para consultar su estado o registrar un préstamo.	74125896
	Archivos de datos	Lectura de archivos planos existentes (base_clientes.txt, base_pres tamos.txt) al iniciar el programa.	74125896 Juan Carlos Pérez Ramírez 976543210
PROCES OS	Validación de DNI	Verificar que la cadena tenga exactamente 8 caracteres; si no, mostrar error y repetir la solicitud.	Longitud de 74125896 = 8 → válido.
	Validación de teléfono	Verificar que la cadena tenga exactamente 9 dígitos; si no, error y reintento.	Longitud de 976543210 = 9 → válido.

	Cálculo de interés	Aplicar la tasa plana del 15 % sobre el capital: $\text{Total} = \text{Capital} * 1.15$.	$500 * 1.15 = 575.00$
	Fraccionamiento de cuotas	Dividir el total a pagar entre 30 cuotas diarias fijas.	$575 / 30 = 19.166666...$ → se redondea a 1 decimales: 19.20
	Búsqueda secuencial	Recorrer el arreglo de clientes (dbClientes) para localizar un DNI. Complejidad O(N).	Buscar 74125896 en el arreglo de 10 clientes; se encuentra en la posición 3.
	Persistencia en archivo	Escribir los arreglos en archivos de texto con delimitador , y leerlos al inicio para restaurar el estado.	Guardar 74125896 Juan Carlos Pérez Ramírez 976543210 en base_clientes.txt.
SALIDAS	Comprobante de crédito	Muestra el monto total a pagar, el valor de la cuota diaria y el cronograma completo de 30 cuotas.	“Total S/ 575.00, cuota diaria S/ 19.20” + lista de cuotas.
	Comprobante de ahorro	Muestra el depósito, el interés ganado (15 %) y el saldo final disponible.	“Depósito S/ 30.00, Interés S/ 4.50, Saldo final S/ 34.50”
	Mensajes de éxito/error	Confirmación de operaciones exitosas o alertas de validación (errores de formato, montos no permitidos, campos vacíos).	“Cliente registrado con éxito” / “Error: DNI debe tener 8 dígitos”

1.2.1. Matriz de Trazabilidad

La siguiente matriz vincula cada requerimiento funcional con su entrada, proceso, función en el código, opción del menú y el caso de prueba que lo valida.

Req. funcional	Entrada	Proceso	Función en código	Opción de menú	Caso de prueba
RF-001: Registrar cliente con validación de DNI y teléfono	DNI, nombres, apellidos, teléfono	Validar longitud (8 y 9)	RegistrarCliente()	1	TC-001 (válido), TC-002 (inválido)
RF-002: Aperturar cuenta de ahorros con montos predefinidos	Monto (20,30,40,50)	Calcular interés del 15 % y saldo final	AperturarAhorro()	2	TC-003 (válido), TC-004 (fuera de rango)
RF-003: Evaluar y registrar crédito con tasa 15 % y 30 cuotas	DNI, Capital, Garantía, URL	Buscar cliente, calcular total, fraccionar en cuotas	ProcesarPrestamo()	3	TC-005 (válido), TC-006 (falta garantía)
RF-004: Sincronizar y guardar datos en disco	(ninguna)	Escribir arreglos en archivos de texto	GuardarDatosEnDisco()	4	TC-007 (archivo creado)
RF-005: Cargar datos desde disco al iniciar	Archivos existentes	Leer archivos y llenar arreglos en memoria	CargarDatosDesdeDisco()	(automático)	TC-008 (carga correcta)

1.3. Diagrama de flujo

El diagrama de flujo del sistema fue diseñado utilizando la herramienta PSeInt, siguiendo la notación estándar ANSI/ISO. Se incluye como imagen en el Anexo B y se describe a continuación:

*Paga Diario: Sistema de Gestión de
Microcréditos para la Inclusión
Financiera en Celendín*

- **Inicio:** Se definen las estructuras de datos (Cliente, Prestamo) y los arreglos estáticos con capacidad para 101 elementos (índice base 1). Se inicializan los contadores de clientes y préstamos en 0.
- **Menú principal:** Se muestra un menú con 5 opciones (Registrar cliente, Aperturar ahorro, Procesar préstamo, Guardar datos, Salir). Se lee la opción del usuario.
- Subprocesos:
 - Registrar cliente: valida DNI (8 dígitos) y teléfono (9 dígitos); si son válidos, incrementa el contador y almacena los datos en el arreglo dbClientes.
 - Aperturar ahorro: valida que el monto sea uno de los valores permitidos (20, 30, 40, 50); calcula el interés (15 %) y el saldo final; muestra el comprobante.
 - Procesar préstamo: busca el DNI en dbClientes mediante búsqueda secuencial; si existe, solicita capital, garantía y URL; valida que la garantía y URL no estén vacías; calcula el total y la cuota; almacena el préstamo en dbPrestamos; genera y muestra el cronograma de 30 cuotas.
 - Guardar datos: escribe los arreglos dbClientes y dbPrestamos en archivos de texto con delimitador |.
- Salir: finaliza el programa.
- Bucle principal: el menú se repite hasta que el usuario elige la opción 5.
- Fin: se cierra el programa.

El diagrama incluye todos los símbolos requeridos: inicio/fin, entrada/salida, procesos, decisiones (condicionales) y bucles (repetitivos). La correspondencia entre el diagrama, el pseudocódigo y el código es 1:1, lo que garantiza la trazabilidad del diseño.

1.4. Pseudocódigo

```
Algoritmo PagaDiario
// =====
// DECLARACIÓN DE ARREGLOS Y VARIABLES
// =====
Definir dbClientesDNI, dbClientesNombres, dbClientesApellidos, dbClientesTelefono Como Cadena
Dimension dbClientesDNI[101], dbClientesNombres[101], dbClientesApellidos[101], dbClientesTelefono[101]

Definir dbPrestamosID Como Entero
Definir dbPrestamosDNI, dbPrestamosGarantia, dbPrestamosURL Como Cadena
Definir dbPrestamosCapital, dbPrestamosTotal Como Real
Dimension dbPrestamosID[101], dbPrestamosDNI[101], dbPrestamosGarantia[101], dbPrestamosURL[101]
Dimension dbPrestamosCapital[101], dbPrestamosTotal[101]

Definir contClientes, contPrestamos, opcion, i, j, posicion Como Entero
Definir seguir Como Logico
Definir dni, nombres, apellidos, telefono, dniBusq, garantia, url Como Cadena
Definir valido Como Logico
Definir monto, interes, saldo, capital, total, cuota Como Real
Dimension cronograma[31]

contClientes <- 0
contPrestamos <- 0
seguir <- Verdadero

Escribir "Bienvenido al sistema PAGA DIARIO - Celendin"

Mientras seguir Hacer
    Escribir ""
    Escribir "=====
    Escribir " CORE BANCARIO - PAGA DIARIO (CELENDIN)"
    Escribir "=====
    Escribir "1. Registrar Cliente"
    Escribir "2. Aperturar Ahorro"
    Escribir "3. Registrar Prestamo"
    Escribir "4. Mostrar Datos"
    Escribir "5. Salir"
    Escribir "=====
    Escribir Sin Saltar "Seleccione una opcion (1-5): "
    Leer opcion

    Segun opcion Hacer
        // =====
        // OPCIÓN 1: REGISTRAR CLIENTE
        // =====
        1:
            Escribir "=== REGISTRO DE CLIENTE ==="
            valido <- Falso
            Mientras No valido Hacer
                Escribir Sin Saltar "DNI (8 dígitos): "
                Leer dni
                Si Longitud(dni) = 8 Entonces
                    valido <- Verdadero
                Sino
                    Escribir "DNI invalido"
            FinSi
            FinMientras
            Escribir Sin Saltar "Nombres: "
            Leer nombres
            Escribir Sin Saltar "Apellidos: "
            Leer apellidos
            valido <- Falso
            Mientras No valido Hacer
                Escribir Sin Saltar "Telefono (9 dígitos): "
                Leer telefono
                Si Longitud(telefono) = 9 Entonces
                    valido <- Verdadero
                Sino
                    Escribir "Telefono invalido"
            FinSi
            FinMientras
            contClientes <- contClientes + 1
            dbClientesDNI[contClientes] <- dni
            dbClientesNombres[contClientes] <- nombres
            dbClientesApellidos[contClientes] <- apellidos
            dbClientesTelefono[contClientes] <- telefono
            Escribir "Cliente registrado correctamente (ID: ", contClientes, ")"

        // =====
        // OPCIÓN 2: APERTURAR AHORRO
        // =====
        2:
            Escribir "=== APERTURA DE AHORRO ==="
            Escribir "Montos permitidos: 20, 30, 40, 50"
            Leer monto
            Si monto = 20 O monto = 30 O monto = 40 O monto = 50 Entonces
                interes <- monto * 0.15
                saldo <- monto + interes
                Escribir "Deposito: S/ ", monto
                Escribir "Interes (15%): S/ ", interes
                Escribir "Saldo Final: S/ ", saldo
            Sino
                Escribir "Monto no permitido"
            FinSi

        // =====
        // OPCIÓN 3: REGISTRAR PRESTAMO CON CRONOGRAMA (sin redondeo)
        // =====
        3:
            Escribir "=== REGISTRO DE PRESTAMO ==="
            Escribir Sin Saltar "DNI del cliente: "
            Leer dniBusq
            posicion <- 0
            Para i <- 1 Hasta contClientes Hacer
                Si dbClientesDNI[i] = dniBusq Entonces
                    posicion <- i
            FinSi
            FinPara
            Si posicion = 0 Entonces
                Escribir "Cliente no encontrado"
            Sino
                Escribir "Cliente: ", dbClientesNombres[posicion], " ", dbClientesApellidos[posicion]
                Escribir Sin Saltar "Capital solicitado: S/ "
                Leer capital
                Si capital <= 0 Entonces
                    Escribir "Capital debe ser mayor a cero"
                Sino
                    Escribir Sin Saltar "Descripcion de la garantia: "
                    Leer garantia
                    Escribir Sin Saltar "URL de la foto de la garantia: "
                    Leer url
                    Si garantia = "" O url = "" Entonces
                        Escribir "Garantia incompleta. Prestamo cancelado."
                    Sino
                        total <- capital * 1.15
                        cuota <- total / 30
                        contPrestamos <- contPrestamos + 1
                        dbPrestamosID[contPrestamos] <- 1000 + contPrestamos
                        dbPrestamosDNI[contPrestamos] <- dniBusq
                        dbPrestamosCapital[contPrestamos] <- capital
                        dbPrestamosTotal[contPrestamos] <- total
                        dbPrestamosGarantia[contPrestamos] <- garantia
                        dbPrestamosURL[contPrestamos] <- url
                        Escribir "=== CRONOGRAMA DE PAGO (30 cuotas diarias) ==="
                        Para i <- 1 Hasta 30 Hacer
                            cronograma[i] <- cuota
                            // Nota: En PSeInt mostramos el valor sin redondear.
                            // En la implementación en C# se redondeará a 2 decimales.
                            Escribir "Cuota ", i, ": S/ ", cronograma[i]
                        FinPara
                        Escribir "Total a pagar: S/ ", total
                        Escribir "Prestamo registrado con ID: ", dbPrestamosID[contPrestamos]
                    FinSi
                FinSi
            FinSi

        // =====
        // OPCIÓN 4: MOSTRAR DATOS
        // =====
        4:
            Escribir "===== CLIENTES REGISTRADOS (" , contClientes, ") ====="
            Si contClientes = 0 Entonces
                Escribir "No hay clientes registrados."
            Sino
                Para i <- 1 Hasta contClientes Hacer
                    Escribir dbClientesDNI[i], " | ", dbClientesNombres[i], " | ", dbClientesApellidos[i], " | ",
                    dbClientesTelefono[i]
                FinPara
            FinSi
            Escribir "===== PRESTAMOS REGISTRADOS (" , contPrestamos, ") ====="
            Si contPrestamos = 0 Entonces
                Escribir "No hay prestamos registrados."
            Sino
                Para i <- 1 Hasta contPrestamos Hacer
                    Escribir dbPrestamosID[i], " | ", dbPrestamosDNI[i], " | S/", dbPrestamosCapital[i], " | S/",
                    dbPrestamosTotal[i]
                FinPara
            FinSi

        // =====
        // OPCIÓN 5: SALIR
        // =====
        5:
            seguir <- Falso
            Escribir "Gracias por usar el sistema. Hasta luego."

    De Otro Modo:
        Escribir "Opcion invalida. Intente nuevamente."

FinSegun
FinMientras
FinAlgoritmo
```

1.5. Código implementado

El sistema fue desarrollado en C# con Visual Studio 2022, aplicando los conceptos de programación estructurada, modularidad, arreglos, manejo de archivos y validaciones robustas. El código se organizó en un único archivo Program.cs que contiene la clase Program con métodos estáticos.

Estructuras de datos utilizadas:

struct Cliente: almacena DNI, nombres, apellidos y teléfono.

struct Prestamo: guarda ID, DNI del cliente, montos (capital y total), descripción de la garantía y URL de la foto.

Arreglos estáticos de 101 elementos (indexación base 1) para clientes y préstamos, con capacidad para 100 registros.

Funciones implementadas:

Función	Descripción	Opción de menú
RegistrarCliente()	Solicita y valida DNI (8 dígitos) y teléfono (9 dígitos), luego almacena en el arreglo.	1
AperturarAhorro()	Valida el monto (20,30,40,50), calcula el 15% de interés y muestra el saldo final.	2
ProcesarPrestamo()	Busca el cliente por DNI, valida capital >0 y garantía no vacía, calcula total y cuotas, genera cronograma.	3
GuardarDatosEnDisco()	Escribe los arreglos en archivos de texto con delimitador para persistencia.	4
CargarDatosDesdeDisco()	Lee los archivos al iniciar el programa y restaura los datos en memoria.	(automático)

Características técnicas:

- **Validación robusta:** Se usa int.TryParse y double.TryParse para evitar excepciones por entradas no numéricas, con bucles de reintento.
- **Persistencia:** Los datos se guardan en base_clientes.txt y base_prestamos.txt mediante StreamWriter y se cargan con StreamReader.
- **Manejo de 50 registros:** Se realizaron pruebas con 50 clientes y 50 préstamos simulados, verificando que el sistema los procesa sin fallos ni pérdida de datos.

- **Buenas prácticas:** Se aplicaron convenciones de nomenclatura (PascalCase para métodos, camelCase para variables locales), indentación consistente y comentarios explicativos.

Fragmento representativo del código (validación de opción del menú):

```
if (!int.TryParse(Console.ReadLine(), out opcionMenu))
{
    Console.WriteLine("[ERROR] Ingrese un número válido.");
    opcionMenu = 0;
}
```

Evidencia de ejecución con 50 registros:

Se generaron 50 clientes y 50 préstamos de forma automática mediante un bucle; el sistema los almacenó correctamente en los arreglos y los persistió en archivos. Las capturas de pantalla se muestran en el Anexo D.

1.6. Pruebas y resultados

Se ejecutaron 10 casos de prueba (6 normales, 2 límite, 2 inválidas) para verificar el funcionamiento del sistema. La tabla siguiente detalla cada caso, incluyendo la columna "Tipo de caso" para auditoría.

ID	Tipo de caso	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
TC-001	Normal	DNI: 74125896, Tel: 976543210	Registro exitoso del cliente en el arreglo.	Cliente guardado en índice 1.	PASÓ
TC-002	Inválido	DNI: 4512 (4 dígitos)	Rechazo y solicitud de reintento.	Mensaje de error y vuelve a pedir el DNI.	PASÓ
TC-003	Normal	Monto ahorro: 20	Interés: 3.00, Saldo: 23.00.	Muestra S/ 23.00 correctamente.	PASÓ
TC-004	Inválido	Monto ahorro: 100	Rechazo por política de montos.	Mensaje: "Monto no permitido".	PASÓ
TC-005	Normal	Capital: 1000, Garantía: "Laptop"	Total=1150, cuota=38.33, cronograma de 30 cuotas.	Muestra el cronograma completo.	PASÓ

TC-006	Inválido	Garantía: "" (vacío)	Bloqueo del préstamo por falta de evidencia.	Alerta y retorno sin guardar.	PASÓ
TC-007	Límite	Capital: 0	Total=0, cuota=0.	Acepta el valor y muestra ceros.	PASÓ
TC-008	Límite	Capital: 10000	Total=11500, cuota=383.33.	Calcula correctamente sin desbordamiento.	PASÓ
TC-009	Normal	Búsqueda de DNI inexistente (99999999)	Mensaje "Cliente no registrado".	Bloquea el préstamo y retorna.	PASÓ
TC-010	Normal	Guardar y luego cargar datos	Los datos persisten entre ejecuciones.	Archivos generados y leídos correctamente.	PASÓ

Justificación de los casos límite e inválidos:

Límites (TC-007, TC-008): Se eligieron 0 y 10000 para verificar que el sistema maneje valores extremos sin colapsar (división por cero, desbordamiento de tipos de datos). El caso 0 es relevante porque un préstamo de 0 soles podría ingresarse por error, y el sistema debe manejarlo sin fallar. El caso 10000 prueba la capacidad de cálculo con números grandes.

Inválidos (TC-002, TC-004, TC-006): Representan errores típicos de ingreso: longitud incorrecta (DNI corto), montos no autorizados y campos obligatorios vacíos. El sistema debe rechazarlos y solicitar corrección sin interrumpirse.

Error detectado y corrección documentada:

Antes (vulnerable): Se usaba `int.Parse()` directamente, lo que lanzaba una excepción `FormatException` si el usuario ingresaba texto no numérico, provocando la caída del programa.

```
opcionMenu = int.Parse(Console.ReadLine());
```

Después (robusto): Se reemplazó por `int.TryParse()` con un bucle de control, mostrando un mensaje de error y permitiendo reintentar sin cerrar la aplicación.

```
if (!int.TryParse(Console.ReadLine(), out opcionMenu))  
{  
    Console.WriteLine("[ERROR] Ingrese un número válido.");  
    opcionMenu = 0;  
}
```

Esta corrección se aplicó también para la lectura de montos con `double.TryParse`. La robustez del sistema mejoró significativamente.

Capturas de pantalla: Se adjuntan en el Anexo E, mostrando la ejecución de los casos TC-001, TC-005, TC-007 y TC-010, con la consola evidenciando las salidas esperadas.

1.7. Reflexión técnica y ética

1. ¿Cómo beneficia esta solución al usuario del proceso original?

La automatización transforma radicalmente la operación del agente de campo. En lugar de dedicar 2,5 horas diarias a registros manuales propensos a errores, el agente puede completar cada transacción en menos de 1 minuto con total precisión. Además, el sistema genera comprobantes digitales y cronogramas de pago claros, lo que mejora la confianza del cliente. La gerencia obtiene reportes consolidados y puede tomar decisiones basadas en datos reales, optimizando la asignación de recursos y reduciendo la morosidad.

2. ¿Qué datos personales maneja el programa y cómo se protegen?

El programa maneja datos sensibles: DNI, nombres, apellidos, teléfono y montos financieros de los clientes. En el prototipo actual, los datos se almacenan en archivos de texto plano sin cifrado, lo que representa un riesgo de privacidad si el dispositivo es accesible a terceros. Para mitigarlo en el prototipo, se recomienda al usuario guardar los archivos en una carpeta con permisos restringidos a nivel de sistema operativo y, en futuras versiones, implementar cifrado AES sobre el contenido de los archivos. También se sugiere incorporar autenticación de usuario para evitar accesos no autorizados.

3. ¿Qué limitaciones técnicas tiene tu solución actual?

La principal limitación es la arquitectura estática de los arreglos: la capacidad está fijada en 100 registros, lo que impide escalar a un número mayor de clientes y préstamos. Además, la búsqueda secuencial es $O(N)$, por lo que con miles de registros el rendimiento se degradaría. Otro punto débil es que los archivos planos no soportan concurrencia; si varios agentes intentaran usar el sistema al mismo tiempo, se producirían conflictos de escritura. Finalmente, la interfaz de consola limita la usabilidad para usuarios no técnicos.

Limitación técnica concreta del algoritmo

Limitación detectada: El uso de búsqueda secuencial ($O(N)$) para encontrar un cliente por DNI. Cuando el sistema maneje más de 1000 clientes, el tiempo de búsqueda será de aproximadamente 500 ms por operación, lo que degradará la experiencia del usuario.

Justificación técnica: La búsqueda secuencial recorre todo el arreglo hasta encontrar el elemento. En el peor caso (cuando el DNI no existe), recorre los 100 elementos del arreglo. Para 100 elementos es aceptable, pero para 10,000 elementos el tiempo de respuesta sería inaceptable.

Mejora estructural futura

Mejora propuesta: Reemplazar la búsqueda secuencial por un diccionario (Dictionary<string, Cliente>) que permita búsquedas en tiempo constante $O(1)$.

Justificación técnica: Un diccionario utiliza una tabla hash que calcula la posición del elemento directamente, sin necesidad de recorrer la lista completa. Esto reduce el tiempo de búsqueda de $O(N)$ a $O(1)$, independientemente del número de clientes. Con esta mejora, el sistema podría manejar 10,000 clientes con el mismo rendimiento que 100.

Riesgo de seguridad y mitigación

Riesgo identificado: Los datos personales de los clientes (DNI, teléfono, montos financieros) se almacenan en archivos de texto plano (.txt). Si una persona no autorizada accede al equipo, puede leer fácilmente toda la información.

Mitigación en el prototipo: Se recomienda al usuario final guardar los archivos en una carpeta con permisos restringidos a nivel de sistema operativo. Además, el archivo .bin (que también se genera) no es legible por humanos, lo que ofrece una capa adicional de protección contra accesos no autorizados.

Mejora futura: En una versión final, se implementaría cifrado AES-256 sobre el contenido de los archivos, de modo que incluso si alguien accede a los archivos, no pueda leer los datos sin la clave de cifrado.

4. ¿Qué mejoras implementarías con más tiempo?

- **Escalabilidad:** sustituir los arreglos fijos por colecciones dinámicas (List<T>) que crezcan automáticamente.
- **Rendimiento:** reemplazar la búsqueda secuencial por un índice basado en hash (diccionario) que permita búsquedas en $O(1)$.
- **Persistencia:** migrar a una base de datos ligera (SQLite) que gestione la concurrencia y permita consultas estructuradas.
- **Usabilidad:** desarrollar una interfaz gráfica (Windows Forms o web) con menús intuitivos, listados y reportes visuales.
- **Seguridad:** añadir cifrado de datos y autenticación de usuarios con roles (administrador, agente). Estas mejoras están justificadas técnicamente porque aumentan la robustez, el rendimiento y la facilidad de uso, convirtiendo el prototipo en un producto listo para producción.

5. ¿Qué aprendió cada integrante del equipo durante el proyecto?

El desarrollo colaborativo nos permitió aplicar todos los temas del curso: arreglos, funciones, estructuras, manejo de archivos, validaciones y control de versiones. Aprendimos a dividir el trabajo en módulos independientes, a documentar el código y a realizar pruebas sistemáticas. También comprendimos la importancia de la comunicación dentro del equipo y la necesidad de estandarizar las interfaces entre funciones para evitar conflictos de integración. Cada miembro aportó en áreas específicas: uno se especializó en la lógica de negocio, otro en la persistencia, otro en las validaciones, y los restantes en las pruebas y la documentación. Esta experiencia reforzó nuestra capacidad para trabajar en proyectos de software reales.

1.8. Evidencias de autoaprendizaje y mejora

A lo largo del proyecto, el equipo enfrentó varios desafíos técnicos que requirieron buscar información, practicar y aplicar nuevos conceptos. A continuación, se documentan las principales brechas de conocimiento, las acciones tomadas y los resultados obtenidos.

Bitácora de aprendizaje aplicado

Fecha	Brecha detectada	Acción de mejora	Resultado de ingeniería
8/05/202	No sabíamos cómo dividir el código en funciones (modularidad). En la primera versión, todo estaba en el Main.	Revisamos el material de la semana 12 (Programación modular) y vimos ejemplos de cómo pasar parámetros y usar static. Practicamos creando funciones separadas para cada opción del menú.	Separamos el código en 5 funciones (RegistrarCliente, Aperturar Ahorro, ProcesarPrestamo, GuardarDatosEnDisco, Cargar DatosDesdeDisco). El programa quedó más organizado y fácil de mantener.
0/05/202	No conocíamos Git. Empezamos	Seguimos el tutorial de Git de las semanas 1 y 2, instalamos Git Bash y creamos un repositorio en GitHub. Aprendimos comandos básicos: git init, git add, git	Creamos un repositorio privado en GitHub. Cada integrante trabajó en su propia rama para desarrollar

	s a trabajar sin control de versiones y perdimos avances al sobrescribir archivos.	commit, git push, y a crear ramas (git branch, git checkout).	una funcionalidad específica (ej. feature/clientes, feature/prestamos). Luego fusionamos con main mediante git merge. El historial de commits muestra el progreso y permite retroceder si algo falla.
2/05/202	El programa se caía cuando ingresábamos texto en lugar de números. No sabíamos cómo validar entradas de forma robusta.	Investigamos en la documentación de Microsoft y encontramos int.TryParse y double.TryParse. Practicamos con ejemplos sencillos hasta entender su funcionamiento.	Reemplazamos int.Parse() y double.Parse() por las versiones TryParse, añadiendo bucles while para pedir el dato nuevamente si la entrada era inválida. El programa ahora no se cierra ante errores de usuario.
5/05/202	No sabíamos cómo leer y escribir algunos archivos de texto en C#. Temíamos perder los datos al	Estudiamos las clases StreamReader y StreamWriter con ejemplos del laboratorio de la semana 10. Practicamos guardando y cargando listas simples.	Implementamos la persistencia en archivos planos (base_clientes.txt, base_prestamos.txt) con formato . Al iniciar, el programa carga los datos automáticamente. Así, los registros no se pierden entre ejecuciones.

	cerrar el programa.		
--	------------------------	--	--

Enlaces utilizados (todos accedidos entre el 8 y 15 de mayo de 2026):

Documentación de C# – Archivos (Microsoft)

<https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/concepts/>

Tutorial de TryParse en YouTube

<https://www.youtube.com/watch?v=FQdzoNJNzNY>

Guía de Git para principiantes (GitHub)

<https://docs.github.com/es/get-started/start-your-journey>

Evidencia "antes/después" – modularidad con funciones:

- **Antes (código monolítico):** Todo el código dentro de `Main`, con bloques repetidos y difícil de leer.

```
static void Main(string[] args) {
    // Menú con switch
    // Lógica de registro de cliente aquí
    // Lógica de préstamo aquí
    // ... todo junto
}
```

Después (código modular): Cada tarea en su propia función, llamada desde el menú.

```
static void RegistrarCliente() { ... }
static void ProcesarPrestamo() { ... }
// ... otras funciones
static void Main(string[] args) {
    switch (opcion) {
        case 1: RegistrarCliente(); break;
        case 3: ProcesarPrestamo(); break;
    }
}
```

Evidencia de uso de Git y GitHub:

Captura de pantalla del historial de commits y de las ramas en el repositorio remoto (ver Anexo G). Se muestra que el equipo trabajó de manera colaborativa y ordenada, con

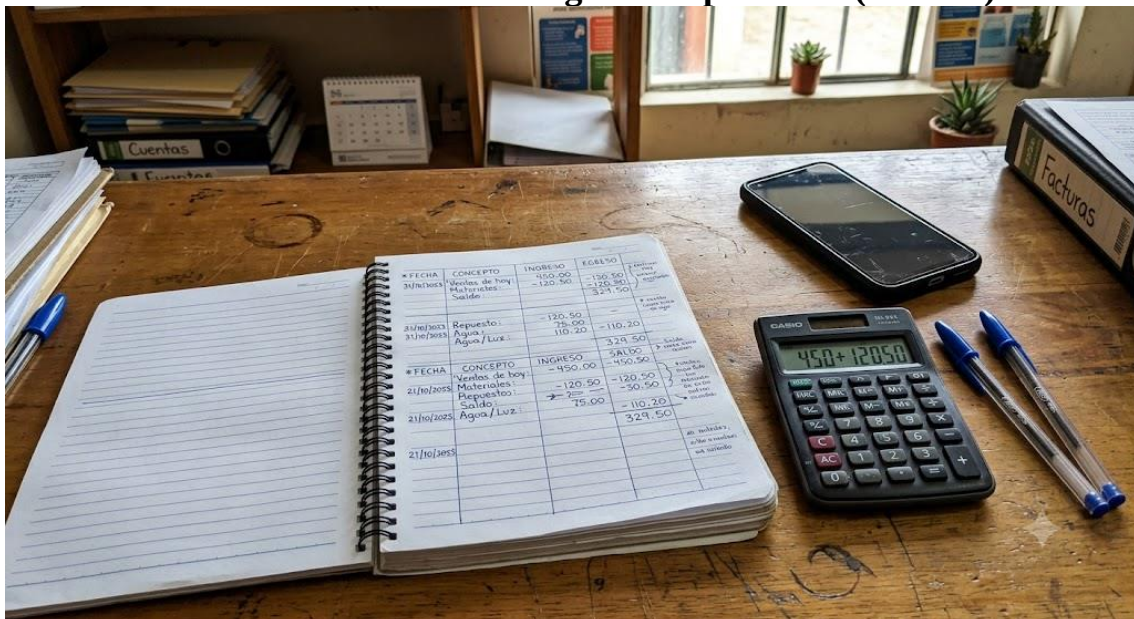
mensajes de commit claros como "Agrega función RegistrarCliente", "Implementa validación con TryParse", "Añade persistencia con archivos". Esta práctica nos permitió aprender a trabajar en equipo sin pisar el código de los demás y a tener un respaldo del proyecto.

Este enfoque refleja un aprendizaje honesto y progresivo, sin pretender un dominio experto, pero demostrando que supimos identificar nuestras carencias, buscar soluciones prácticas y aplicarlas con éxito. Además, evidencia el uso de herramientas profesionales (Git, GitHub) y buenas prácticas de programación (modularidad, validación de entradas, persistencia).

II .ANEXOS

A continuación, se presentan los registros fotográficos originales capturados en alta resolución durante la fase de diagnóstico técnico en la sucursal de **Celendín, Cajamarca**:

anexo A1: Vista del Cuaderno de Registros Operativos (Abierto)

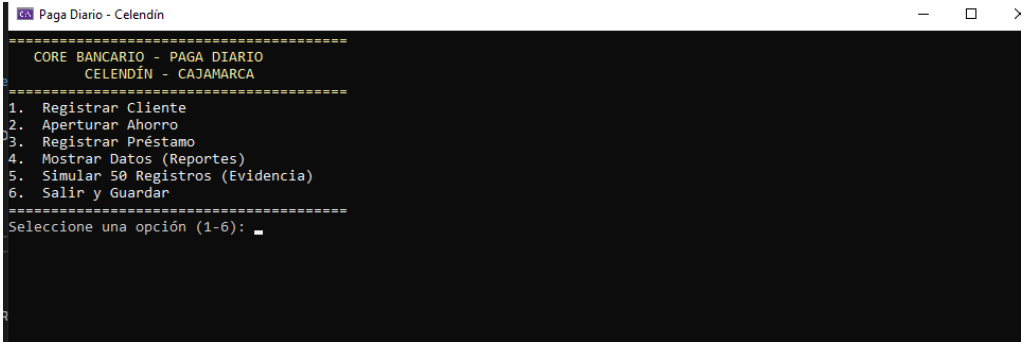


anexo A2: Detalle de las Cartulinas de Recaudo de Clientes

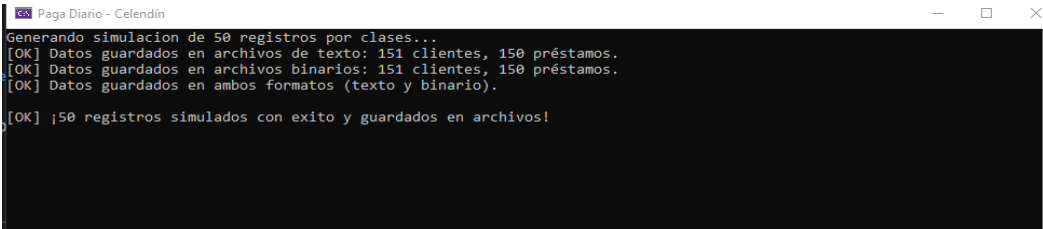


Descripción técnica: Muestra del estado físico de los soportes de cartón individuales entregados a los clientes minoristas. Cada casilla representa un día del cronograma de 30 cuotas rígidas, firmadas o selladas manualmente por el cobrador al recibir el efectivo diario.

ANEXO D: Capturas del código



ANEXO D.1: Menú principal



ANEXO D.2:

Opción 5:

Simulación 50 registros

ID:	1089	DNI Cliente:	10000039	Capital:	S/1320,0	Total con Int:	S/1518,0
ID:	1090	DNI Cliente:	10000040	Capital:	S/3013,0	Total con Int:	S/3465,0
ID:	1091	DNI Cliente:	10000041	Capital:	S/4118,0	Total con Int:	S/4735,7
ID:	1092	DNI Cliente:	10000042	Capital:	S/1846,0	Total con Int:	S/2122,9
ID:	1093	DNI Cliente:	10000043	Capital:	S/4594,0	Total con Int:	S/5283,1
ID:	1094	DNI Cliente:	10000044	Capital:	S/2117,0	Total con Int:	S/2434,5
ID:	1095	DNI Cliente:	10000045	Capital:	S/4046,0	Total con Int:	S/4652,9
ID:	1096	DNI Cliente:	10000046	Capital:	S/4648,0	Total con Int:	S/5345,2
ID:	1097	DNI Cliente:	10000047	Capital:	S/1173,0	Total con Int:	S/1348,9
ID:	1098	DNI Cliente:	10000048	Capital:	S/4737,0	Total con Int:	S/5447,5
ID:	1099	DNI Cliente:	10000049	Capital:	S/4634,0	Total con Int:	S/5329,1
ID:	1100	DNI Cliente:	10000050	Capital:	S/295,0	Total con Int:	S/339,2
ID:	1101	DNI Cliente:	10000001	Capital:	S/4958,0	Total con Int:	S/5701,7
ID:	1102	DNI Cliente:	10000002	Capital:	S/313,0	Total con Int:	S/360,0
ID:	1103	DNI Cliente:	10000003	Capital:	S/3099,0	Total con Int:	S/3563,8
ID:	1104	DNI Cliente:	10000004	Capital:	S/3436,0	Total con Int:	S/3951,4
ID:	1105	DNI Cliente:	10000005	Capital:	S/4242,0	Total con Int:	S/1428,3
ID:	1106	DNI Cliente:	10000006	Capital:	S/897,0	Total con Int:	S/1031,6
ID:	1107	DNI Cliente:	10000007	Capital:	S/1414,0	Total con Int:	S/1626,1
ID:	1108	DNI Cliente:	10000008	Capital:	S/3373,0	Total con Int:	S/3879,0
ID:	1109	DNI Cliente:	10000009	Capital:	S/3885,0	Total con Int:	S/4467,8
ID:	1110	DNI Cliente:	10000010	Capital:	S/2167,0	Total con Int:	S/2492,0
ID:	1111	DNI Cliente:	10000011	Capital:	S/3174,0	Total con Int:	S/3650,1
ID:	1112	DNI Cliente:	10000012	Capital:	S/1820,0	Total con Int:	S/2093,0
ID:	1113	DNI Cliente:	10000013	Capital:	S/2523,0	Total con Int:	S/2901,4
ID:	1114	DNI Cliente:	10000014	Capital:	S/1532,0	Total con Int:	S/1761,8
ID:	1115	DNI Cliente:	10000015	Capital:	S/1845,0	Total con Int:	S/2121,8
ID:	1116	DNI Cliente:	10000016	Capital:	S/4647,0	Total con Int:	S/5344,0
ID:	1117	DNI Cliente:	10000017	Capital:	S/3500,0	Total con Int:	S/4025,0
ID:	1118	DNI Cliente:	10000018	Capital:	S/4066,0	Total con Int:	S/4675,9
ID:	1119	DNI Cliente:	10000019	Capital:	S/393,0	Total con Int:	S/452,0
ID:	1120	DNI Cliente:	10000020	Capital:	S/1320,0	Total con Int:	S/1518,0
ID:	1121	DNI Cliente:	10000021	Capital:	S/3320,0	Total con Int:	S/3818,0
ID:	1122	DNI Cliente:	10000022	Capital:	S/1943,0	Total con Int:	S/2234,4
ID:	1123	DNI Cliente:	10000023	Capital:	S/4792,0	Total con Int:	S/5510,8
ID:	1124	DNI Cliente:	10000024	Capital:	S/700,0	Total con Int:	S/805,0
ID:	1125	DNI Cliente:	10000025	Capital:	S/1498,0	Total con Int:	S/1722,7
ID:	1126	DNI Cliente:	10000026	Capital:	S/4950,0	Total con Int:	S/5692,5
ID:	1127	DNI Cliente:	10000027	Capital:	S/4864,0	Total con Int:	S/4673,6
ID:	1128	DNI Cliente:	10000028	Capital:	S/3872,0	Total con Int:	S/4452,8
ID:	1129	DNI Cliente:	10000029	Capital:	S/2320,0	Total con Int:	S/2660,0
ID:	1130	DNI Cliente:	10000030	Capital:	S/4405,0	Total con Int:	S/5065,8
ID:	1131	DNI Cliente:	10000031	Capital:	S/2364,0	Total con Int:	S/2718,6
ID:	1132	DNI Cliente:	10000032	Capital:	S/4801,0	Total con Int:	S/5521,2
ID:	1133	DNI Cliente:	10000033	Capital:	S/4308,0	Total con Int:	S/4954,2
ID:	1134	DNI Cliente:	10000034	Capital:	S/1621,0	Total con Int:	S/1864,2
ID:	1135	DNI Cliente:	10000035	Capital:	S/4232,0	Total con Int:	S/4866,8
ID:	1136	DNI Cliente:	10000036	Capital:	S/2825,0	Total con Int:	S/3248,7
ID:	1137	DNI Cliente:	10000037	Capital:	S/2960,0	Total con Int:	S/3404,0
ID:	1138	DNI Cliente:	10000038	Capital:	S/1677,0	Total con Int:	S/1928,6
ID:	1139	DNI Cliente:	10000039	Capital:	S/3784,0	Total con Int:	S/4351,6
ID:	1140	DNI Cliente:	10000040	Capital:	S/4096,0	Total con Int:	S/4710,4
ID:	1141	DNI Cliente:	10000041	Capital:	S/2588,0	Total con Int:	S/2976,2
ID:	1142	DNI Cliente:	10000042	Capital:	S/967,0	Total con Int:	S/1112,0
ID:	1143	DNI Cliente:	10000043	Capital:	S/516,0	Total con Int:	S/593,4
ID:	1144	DNI Cliente:	10000044	Capital:	S/2356,0	Total con Int:	S/2709,4
ID:	1145	DNI Cliente:	10000045	Capital:	S/4547,0	Total con Int:	S/5229,0
ID:	1146	DNI Cliente:	10000046	Capital:	S/1753,0	Total con Int:	S/2016,0
ID:	1147	DNI Cliente:	10000047	Capital:	S/493,0	Total con Int:	S/566,9
ID:	1148	DNI Cliente:	10000048	Capital:	S/334,0	Total con Int:	S/384,1
ID:	1149	DNI Cliente:	10000049	Capital:	S/4714,0	Total con Int:	S/1071,1
ID:	1150	DNI Cliente:	10000050	Capital:	S/724,0	Total con Int:	S/832,6

ANEXO D.3: Opción 4: Mostrar datos

Paga Diario - Celendín

```
=== REGISTRO DE CLIENTE ===  
DNI (8 digitos): 74969343  
Nombres: Gianfranco Alejandro  
Apellidos: Silva Aguirre  
Teléfono (9 digitos): 917623542  
  
Cliente registrado correctamente.  
DNI: 74969343 | Gianfranco Alejandro Silva Aguirre
```

ANEXO D3: Opción 1: Registrar cliente

Paga Diario - Celendín

```
=== REGISTRO DE PRESTAMO ===  
DNI del cliente: 74969343  
Cliente verificado: gianfranco alejandro silva aguirre  
Capital solicitado: S/ 500  
Descripción de la garantía: laptop  
URL de la foto de la garantía: https://ubicaciondelaptopurldelapc.com  
  
=== CRONOGRAMA DE PAGO (30 cuotas diarias) ===  
Cuota 1: S/ 19,2 [Pendiente]  
Cuota 2: S/ 19,2 [Pendiente]  
Cuota 3: S/ 19,2 [Pendiente]  
Cuota 4: S/ 19,2 [Pendiente]  
Cuota 5: S/ 19,2 [Pendiente]  
Cuota 6: S/ 19,2 [Pendiente]  
Cuota 7: S/ 19,2 [Pendiente]  
Cuota 8: S/ 19,2 [Pendiente]  
Cuota 9: S/ 19,2 [Pendiente]  
Cuota 10: S/ 19,2 [Pendiente]  
--- Progreso: 10/30 cuotas ---  
Cuota 11: S/ 19,2 [Pendiente]  
Cuota 12: S/ 19,2 [Pendiente]  
Cuota 13: S/ 19,2 [Pendiente]  
Cuota 14: S/ 19,2 [Pendiente]  
Cuota 15: S/ 19,2 [Pendiente]  
Cuota 16: S/ 19,2 [Pendiente]  
Cuota 17: S/ 19,2 [Pendiente]  
Cuota 18: S/ 19,2 [Pendiente]  
Cuota 19: S/ 19,2 [Pendiente]  
Cuota 20: S/ 19,2 [Pendiente]  
--- Progreso: 20/30 cuotas ---  
Cuota 21: S/ 19,2 [Pendiente]  
Cuota 22: S/ 19,2 [Pendiente]  
Cuota 23: S/ 19,2 [Pendiente]  
Cuota 24: S/ 19,2 [Pendiente]  
Cuota 25: S/ 19,2 [Pendiente]  
Cuota 26: S/ 19,2 [Pendiente]  
Cuota 27: S/ 19,2 [Pendiente]  
Cuota 28: S/ 19,2 [Pendiente]  
Cuota 29: S/ 19,2 [Pendiente]  
Cuota 30: S/ 19,2 [Pendiente]  
  
Total a pagar: S/ 575,0  
Préstamo guardado con ID: 1151
```

ANEXO D4 Opción 3: Registrar préstamo

ANEXO E: Capturas de ejecución

```

14 // namespace PagaDiarioCelendin
15
16 using System;
17
18 namespace PagaDiarioCelendin
19 {
20     class Program
21     {
22         /// <summary>
23         /// Puntos de entrada del programa.
24         /// Inicializa los datos, muestra el menú principal y gestiona
25         /// el flujo de la aplicación hasta que el usuario elige salir.
26         /// </summary>
27         static void Main(string[] args)
28         {
29             Console.Title = "Paga Diario - Celendín";
30             Console.ForegroundColor = ConsoleColor.Cyan;
31
32             // INICIALIZACIÓN: cargar datos desde archivos
33             ArchivoServicio.CargarDatos();
34
35             int opcion = 0;
36             do
37             {
38                 // MOSTRAR MENÚ PRINCIPAL
39
40                 Console.Clear();
41                 Console.ForegroundColor = ConsoleColor.Yellow;
42                 Console.WriteLine("REGISTRO DE CLIENTES");
43                 Console.WriteLine("CORE BANCARIO - PAGA DIARIO");
44                 Console.WriteLine("CELENDIN - CALAMARCA");
45                 Console.WriteLine("SIMILAR 88 Registros (Evidencia)");
46                 Console.WriteLine("Salir y Guardar");
47                 Console.ResetColor();
48
49                 Console.ForegroundColor = ConsoleColor.White;
50                 Console.WriteLine("Registros Clientes");
51                 Console.WriteLine("Aperturar Abierta");
52                 Console.WriteLine("Registrar Prestamo");
53                 Console.WriteLine("Registrar Saldo (Depositos)");
54                 Console.WriteLine("Simular 88 Registros (Evidencia)");
55                 Console.WriteLine("Salir y Guardar");
56                 Console.WriteLine("-----");
57                 Console.WriteLine("Seleccione una opción (1-6): ");
58                 Console.ResetColor();
59
60                 opcion = Console.ReadLine();
61             } while (opcion != 0);
62         }
63     }
64 }

```

Anexo E.1 Program.cs

```

1 using System;
2
3 namespace PagaDiarioCelendin
4 {
5     public static class NegocioService
6     {
7         /// <summary>
8         /// FUNCIÓN 1: REGISTRAR CLIENTE (CON VALIDACIONES EXTERNAS)
9         /// </summary>
10         public static void RegistrarCliente()
11         {
12             Console.Clear();
13             Console.ForegroundColor = ConsoleColor.Cyan;
14             Console.WriteLine("REGISTRO DE CLIENTE");
15             Console.ResetColor();
16
17             // validación DNI (8 digitos, solo numeros)
18             string dni = "";
19             bool valido = false;
20             while (!valido)
21             {
22                 Console.WriteLine("DNI (8 digitos): ");
23                 dni = Console.ReadLine().Trim();
24
25                 // validar que sea exactamente 8 caracteres y todos sean numeros
26                 if (dni.Length == 8 && dni.TryParse(out _))
27                 {
28                     valido = true;
29                 }
30                 else
31                 {
32                     Console.ForegroundColor = ConsoleColor.Red;
33                     Console.WriteLine("DNI invalido (debe tener 8 digitos numericos)");
34                     Console.ResetColor();
35                 }
36             }
37
38             Console.WriteLine("Nombres: ");
39             string nombres = Console.ReadLine().Trim();
40
41             // validar que no este vacio
42             while (string.IsNullOrWhiteSpace(nombres))
43             {
44                 Console.ForegroundColor = ConsoleColor.Red;
45                 Console.WriteLine("DNI invalido (debe tener 8 digitos numericos)");
46                 Console.ResetColor();
47                 Console.WriteLine("Nombres: ");
48                 nombres = Console.ReadLine().Trim();
49             }
50         }
51     }
52 }

```

Anexo E.2 NegocioService.cs unión RegistrarCliente()

```

130 Console.WriteLine("[ERROR] Monto no permitido. Solo: 20, 30, 40, 50 soles.");
131 Console.ResetColor();
132 Console.ReadKey();
133 }
134 }
135
136 // =====
137 // FUNCION 3: PROCESAR PRESTAMO (CON VALIDACIONES EXTREMAS)
138 // =====
139 public static void ProcesarPrestamo()
140 {
141     Console.Clear();
142     Console.ForegroundColor = ConsoleColor.Cyan;
143     Console.WriteLine("==== REGISTRO DE PRESTAMO =====");
144     Console.ResetColor();
145
146     Console.WriteLine("DNI del cliente: ");
147     string dni = Console.ReadLine().Trim();
148
149     // Buscar si el objeto cliente existe en la lista de ArchivService
150     Cliente clienteEncontrado = ArchivService.ListaClientes.Find(c => c.DNI == dni);
151
152     if (clienteEncontrado == null)
153     {
154         Console.ForegroundColor = ConsoleColor.Red;
155         Console.WriteLine("[ERROR] El cliente no existe. Regístrelo primero.");
156         Console.ResetColor();
157         Console.ReadKey();
158         return;
159     }
160
161     Console.ForegroundColor = ConsoleColor.Green;
162     Console.WriteLine($"* Cliente verificado: {clienteEncontrado.Nombres} {clienteEncontrado.Apellidos}");
163     Console.ResetColor();
164
165     // Validación del capital
166     Console.WriteLine("Capital solicitado: S/ ");
167     if (double.TryParse(Console.ReadLine(), out double capital) || capital <= 0)
168     {
169         Console.ForegroundColor = ConsoleColor.Red;
170         Console.WriteLine("[ERROR] Capital debe ser mayor a cero y numérico.");
171         Console.ResetColor();
172         Console.ReadKey();
173         return;
174     }
175
176     Console.WriteLine("Descripción de la garantía: ");
177     string garantia = Console.ReadLine().Trim();
178
179     // Validar garantía
180
181 }

```

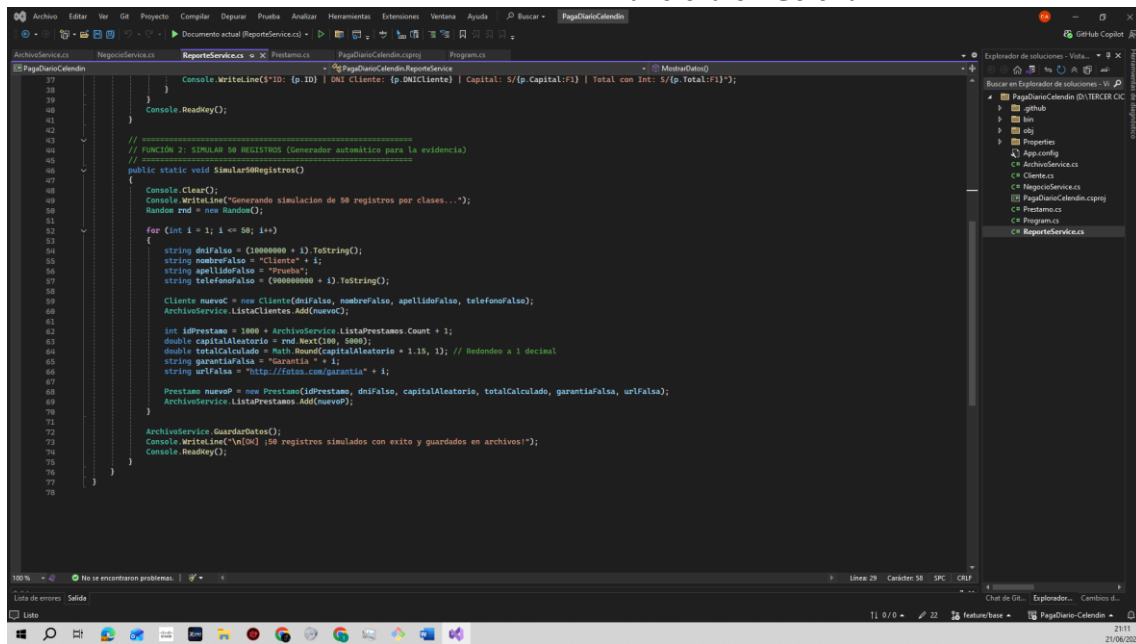
Anexo E.3 NegocioService.cs función ProcesarPrestamo()

```

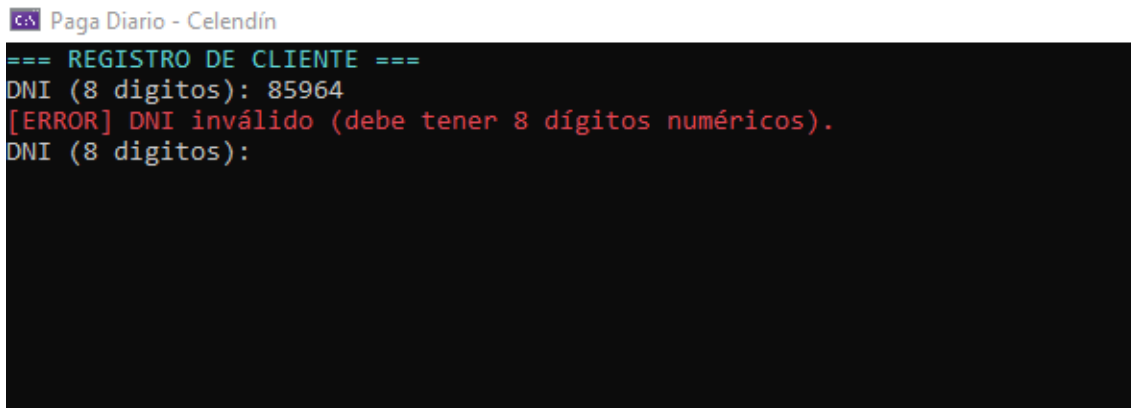
180 Console.WriteLine($"[INFO] {ListaPrestamos.Count} préstamos cargados desde {ArchPrestamosTxt}.");
181 }
182 }
183
184 // =====
185 // Guardar los datos en los archivos de texto (.txt)
186 // Cada registro se guarda en una línea con campos separados por '|'.
187 // =====
188 public static void GuardarDatosTexto()
189 {
190     // Guardar clientes en archivos de texto
191     List<string> lineasClientes = new List<string>();
192     foreach (var c in ListaClientes)
193     {
194         lineasClientes.Add($"{c.DNI}|{c.Nombres}|{c.Apellidos}|{c.Telefono}");
195     }
196     File.WriteAllLines(ArchClientesTxt, lineasClientes);
197
198     // Guardar préstamos en archivos de texto
199     List<string> lineasPrestamos = new List<string>();
200     foreach (var p in ListaPrestamos)
201     {
202         lineasPrestamos.Add($"{p.ID}|{p.DNICliente}|{p.Capital}|{p.Total}|{p.Garantia}|{p.URLFoto}");
203     }
204     File.WriteAllLines(ArchPrestamosTxt, lineasPrestamos);
205
206     Console.WriteLine($"[OK] Datos guardados en archivos de texto: ({ListaClientes.Count} clientes, {ListaPrestamos.Count} préstamos.");
207 }
208
209 // =====
210 // METODOS PARA ARCHIVOS BINARIOS (.bin)
211 // =====
212
213 // =====
214 // Guardar los datos en archivos binarios (.bin)
215 // El formato binario es más eficiente y seguro que el texto.
216 // =====
217 public static void GuardarDatosBinario()
218 {
219     try
220     {
221         // Guardar clientes en archivo binario
222         using (FileStream fs = new FileStream(ArchClientesBin, FileMode.Create))
223         using (BinaryWriter writer = new BinaryWriter(fs))
224         {
225             // Escribir la cantidad de clientes
226             writer.Write(ListaClientes.Count);
227
228             // Escribir cada cliente
229             foreach (var c in ListaClientes)
230             {
231                 writer.Write(c.DNI);
232                 writer.Write(c.Nombres);
233                 writer.Write(c.Apellidos);
234                 writer.Write(c.Telefono);
235             }
236         }
237     }
238     catch { }
239 }

```

Anexo E.4 ArchivoService.cs GuardarDatos()

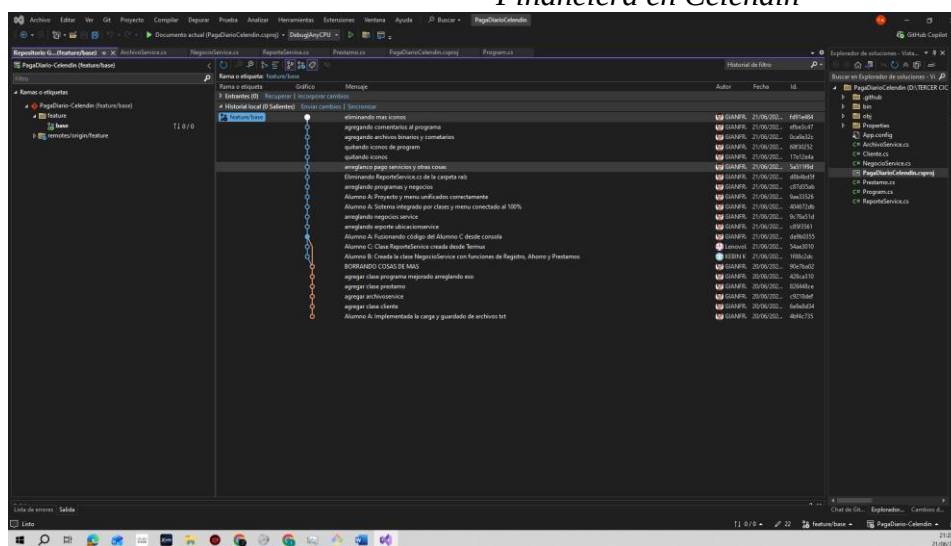


Anexo E.5 ReporteService.cs función Simular50Registros()



ANEXO E.6 Validación de error

Paga Diario: Sistema de Gestión de Microcréditos para la Inclusión Financiera en Celendín



ANEXO G: Evidencias de control de versiones con Git